| DS | **Session** | Graph | **Question Information** | Level 1 Challenge 89 |
|---|---|---|---|---|

**Question description**

A game consists of n rooms and m teleporters. At the beginning of each day, you start in room 1 and you have to reach room n.
You can use each teleporter at most once during the game. How many days can you play if you choose your routes optimally?

**Constraints**

$2 \le n \le 500$

$1 \le m \le 1000$

$1 \le a, b \le n$

**Input**

The first input line has two integers n and m: the number of rooms and teleporters. The rooms are numbered 1,2,…,n.
After this, there are m lines describing the teleporters. Each line has two integers a and b: there is a teleporter from room a to room b.
There are no two teleporters whose starting and ending room are the same.

**Output**

First print an integer k: the maximum number of days you can play the game. Then, print k route descriptions according to the example. You can print any valid solution.

Logical Test Cases

Test Case 1

Test Case 2

INPUT (STDIN)

```
6 7
1 2
1 3
2 6
3 4
3 5
4 6
5 6
```

EXPECTED OUTPUT

```
2
4
1 3 5 6
3
1 2 6
```

INPUT (STDIN)

```
6 7
1 4
3 5
4 6
5 6
2 2
1 3
2 6
```

EXPECTED OUTPUT

```
2
4
1 3 5 6
3
1 4 6
```

## Mandatory Test Cases

### Test Case 1

KEYWORD

```
int bfs(int n,int s,int t)
```

## Complexity Test Cases

### Test Case 1

CYCLOMATIC COMPLEXITY

```
7
```

### Test Case 2

TOKEN COUNT

```
920
```

### Test Case 3

NLOC

```
110
```

# Code

teleporters_routes.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

#define MAXN 505
#define MAXM 2005

typedef struct {
    int to, cap, flow, rev;
} Edge;

Edge edges[2 * MAXM];
int head[MAXN], next_edge[2 * MAXM], edge_cnt = 0;

void add_edge(int from, int to, int cap) {
    edges[edge_cnt] = (Edge){to, cap, 0, edge_cnt + 1};
    next_edge[edge_cnt] = head[from];
    head[from] = edge_cnt++;

    edges[edge_cnt] = (Edge){from, 0, 0, edge_cnt - 1};
    next_edge[edge_cnt] = head[to];
    head[to] = edge_cnt++;
}

int parent_edge[MAXN];

int bfs(int n, int s, int t) {
    for (int i = 1; i <= n; i++) parent_edge[i] = -1;
    int q[MAXN], qh = 0, qt = 0;
    q[qt++] = s;
    parent_edge[s] = -2;

    while (qh < qt) {
        int u = q[qh++];
        if (u == t) return 1;
        for (int e = head[u]; e != -1; e = next_edge[e]) {
            int v = edges[e].to;
            if (parent_edge[v] == -1 && edges[e].cap - edges[e].flow > 0) {
                parent_edge[v] = e;
                q[qt++] = v;
            }
        }
    }
    return 0;
}

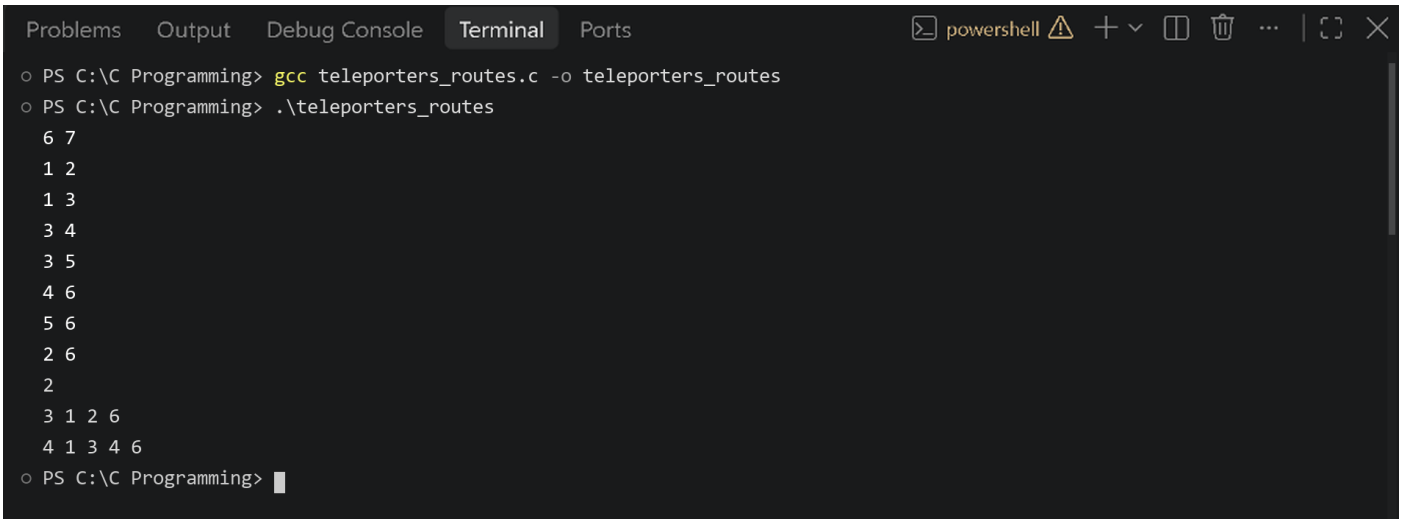
int main() {
    int n, m;
    if (scanf("%d %d", &n, &m) != 2) return 0;
    for (int i = 1; i <= n; i++) head[i] = -1;

    for (int i = 0; i < m; i++) {
        int u, v;
        scanf("%d %d", &u, &v);
        add_edge(u, v, 1);
    }

    int max_days = 0;
    while (bfs(n, 1, n)) {
        for (int v = n; v != 1; ) {
            int e = parent_edge[v];
            edges[e].flow += 1;
            edges[edges[e].rev].flow -= 1;
            v = edges[edges[e].rev].to;
        }
        max_days++;
    }

    printf("%d\n", max_days);
    for (int d = 0; d < max_days; d++) {
        int path[MAXN], plen = 0;
        int curr = 1;
        path[plen++] = curr;
        while (curr != n) {
            for (int e = head[curr]; e != -1; e = next_edge[e]) {
                if (edges[e].flow == 1) {
                    edges[e].flow = 0;
                    curr = edges[e].to;
                    path[plen++] = curr;
                    break;
                }
            }
        }
        printf("%d", plen);
        for (int i = 0; i < plen; i++) printf(" %d", path[i]);
        printf("\n");
    }
    return 0;
}
```

## Output


Problems
Output
Debug Console
Terminal
Ports
powershell
PS C:\C Programming> gcc teleporters_routes.c -o teleporters_routes
PS C:\C Programming> .\teleporters_routes
6 7
1 2
1 3
3 4
3 5
4 6
5 6
2 6
2
3 1 2 6
4 1 3 4 6
PS C:\C Programming>